SYSTEM DEVELOPMENT EXERCISE

さんたく

3つの案から、ひとつ選ぶだけ。

AIが3つのタスクを提案し、選ぶだけで実行できるタスク管理アプリ

Spring Boot 4 / Java 21 / MySQL / Gemini Developer API

システム開発演習（20日間）

野口

押すだけで、次の一歩が決まる

① スタートを押す

ボタンは1つだけ。
考えることがない

② AIが3案出す

重さの違う3つ。
今日の自分で選べる

③ 選ぶと始まる

手順つきのタスクになって
リストへ入る

利用者が行うのは**「押す」と「選ぶ」の2つだけ**。タスクを書き出す作業も、どれから手をつけるかの判断も要らない。

企画 —— 背景

タスク管理アプリが続かない理由

既存のアプリ

- 自分でタスクを書き込む
- 自分でどれからやるか決める
- 書いただけで放置される
- 粒度が大きすぎて手が付かない

生成AIに相談する場合

- 毎回、状況を説明し直す
- 良い答えを得るには手間がかかる
- 結果を自分で記録する必要がある
- その手間で結局続かない

> どちらも「書く」「決める」「説明する」が先に立ち、
> **やりたかったことに手が届く前に止まる。**

企画 —— 解決したい課題

止まる場所は、だいたい決まっている

つまずく場面	さんたくでの解き方
何から手を付けるか決められない	AIが3案に絞る。選択肢が3つなら選べる
タスクが大きすぎて着手できない	3〜10分と、集中時間に合わせた長さ。重さを分けて提案する
どこまでやれば終わりか分からない	提案と同時に「おわりの目安」を1行で生成する
登録したまま忘れて溜まる	3案のうち1つは、リストにある未着手タスクから出す
完了ボタンの押し忘れ	着手と同時に完了を予約する「やっちゃう」機能

目的とコンセプト

迷っているより、まず実行。
ほんの少しでも。

目的は「タスクを管理すること」ではなく、**止まっている人が動き出すこと。**

5分でもいい。動けたなら、それが成功。

画面構成

画面は6つ。一本道で進む

S-01 **ログイン** Googleで › S-02 **3つの質問** 初回のみ › S-03 **スタート** ボタン1つ › S-04 **3案の提案** 重さの違う3枚 › S-05 **タスク一覧** 追加・完了 › S-06 **設定** 通知・演出



スタート画面 — 押すものは1つ



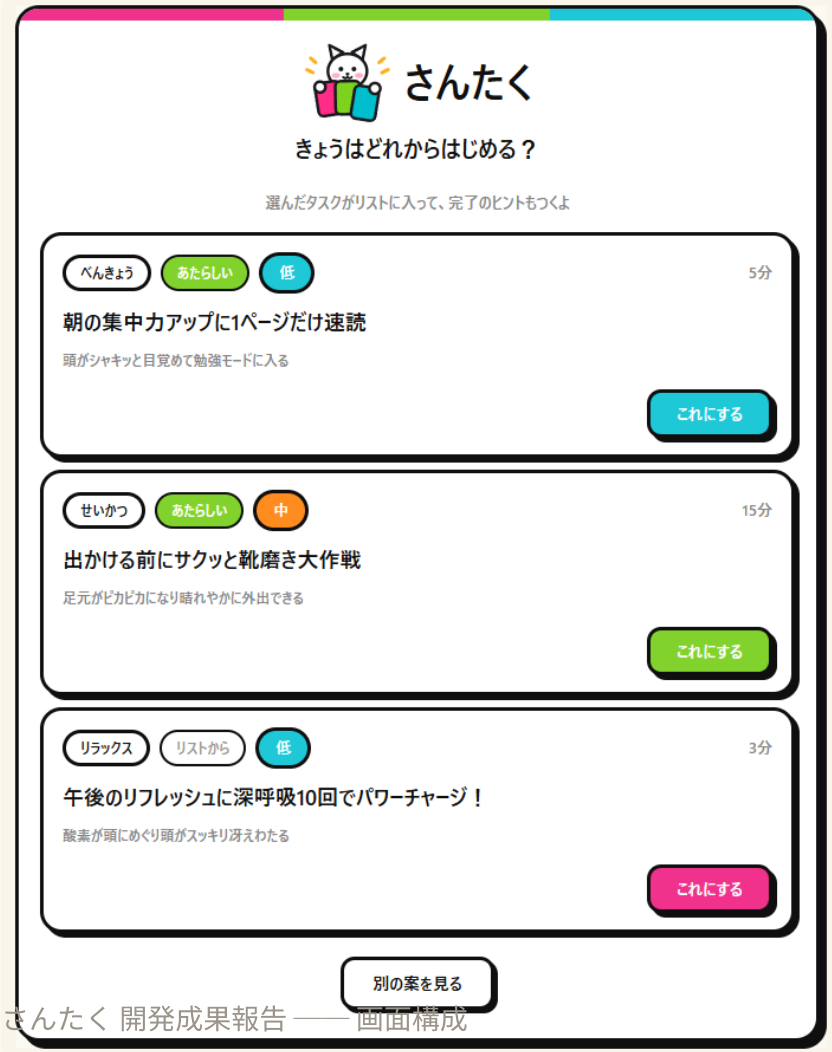
最初の3つの質問（初回のみ）

開いた画面に、押すものを1つだけ置く。 一覧が先に目に入ると、何から手を付けるかを考えることになる。

はじめに3つだけ聞く。 3択が3問で、入力欄はない。答えはそのまま提案の材料になる。

画面構成 —— 提案

3案は、重さが違う



1つ目　3〜10分

今日はもう動けないと思っている人でも手が出るもの

2つ目　集中時間に合わせて

今日中に終わって、進んだと目で分かるもの

3つ目　リストから

登録したまま止まっている自分のタスクを、そのまま出す

3つ目がリストからなのは、押すたびにタスクが増え続けるのを防ぐため。減る経路が完了しかないと、溜まる一方になる。

選ぶか、作り直すかの2択。迷ったら「別の案を見る」で、何度でも作り直せる。

画面構成 —— タスク一覧

選んだあとは、この1画面で回る



主な操作

- 上の欄から追加（難易度・締切も同じ行）
- ドラッグで並べ替え（順番は保存される）
- 鉛筆で編集、× で削除
- 完了したものは下に折りたたむ

やっちゃう（自動完了）

完了のヒントの右上にあるボタン。押すと予約され、時間が来ると自動で完了になり通知が届く。**アプリを閉じていても完了する。**

話したいことは4つ

01

個人情報を持たない

名前もメールも受け取っていない

02

AIに何をさせたか

AIは文章を書くだけ。何を出すかはアプリが決める

03

タスク管理画面

入口を1つにして、迷う場所を作らない

04

続けるための仕組み

続かないのは意志ではなく、仕組みの問題

機能の数ではなく、**持たない・任せすぎない・止めない**を判断の軸にした。

テーマ 01

個人情報を持たない

項目	このアプリでの扱い
名前	受け取らない
メールアドレス	受け取らない
本人を見分ける鍵	Googleが発行する識別子だけ
タスクの中身	持っている。題名はAIにも送る

名前もメールアドレスも、**置く場所そのものが無い。**

「守る」のではなく「最初から持たない」。
漏れるものが無ければ、守り方を考えなくていい

通知はメールではなく Web Push。
送り先はブラウザが発行するURLなので、アドレスを持たずに声をかけられる

AIを使うことを画面に明記。
アプリが持たなくても、**利用者がタスクに書いた個人情報は Google に送られる。**気づかずに書いてしまわないよう、ログイン画面と設定画面に「このアプリ内で入力した内容は Google に送信されます」と書いている

テーマ 02 —— AIに何をさせたか

外に出るのは、1か所だけ

アプリ側

分野と時間を決める

時間帯・曜日・3つの答え・完了率から、1件目と2件目の分野を決める

アプリ側

3件目のタスクを選ぶ

リストに残っているものから1件。かんたん順／締切順／古い順

アプリ側

AIへの指示文を作る

決めた分野と時間、いま抱えているタスク、出さないものを並べて、1本の指示文にする

Gemini

3案を書く

題名・理由・完了のヒント

アプリ側

確かめて保存

形を確認してカード3枚にし、送った文章ごと記録する

どの分野を出すかも、どのタスクを使うかも、**アプリが先に決めている。**AIが書くのは文章だけ。

AIに送った指示文は、返ってきた3案といっしょに保存している。**あとで突き合わせて、提案の質を高める材料にできる。**

テーマ 02 —— 返す形と、落ちたとき

形を固定する。落ちても止まらない

返す形を固定する

```
"generationConfig", Map.of(
 "responseMimeType", "application/json",
 "responseSchema",  schema(),
 "temperature",     0.8)
```

文章で「JSONで返して」と頼むだけだと、項目名が揺れる・数字が文字で返る・項目が欠ける。**型を宣言すれば固定できる。**

落ちても3案は出る

起きること	対応
429（1分の制限）・503	少し待って3回まで試す
429（1日20回の上限）	試し直さない
403・404	すぐ諦める
どの場合も最後は	固定3案で先へ進む

> 出どころ（AI・固定3案・モック）を記録しているので、**AIが何割失敗したかを後から数えられる。**

テーマ 02 —— 記録を次に返す

使うほど、終わらせている分野が出やすくなる

$$\frac{(\text{終わった数} + 1)}{(\text{出した数} + 3)}$$

分野ごとにこの値を出して、次の提案の点数に足している。

- 選ばれた数ではなく**終わった数**を見る。選ぶだけを評価にすると、選びやすいだけの提案に寄る
- 分母に3、分子に1を足すので、**1回出して1回終わっただけの分野が「100%」で独占しない**
- まだ出していない分野は 0.33 で並ぶので、データが無いうちは差が出ず、場合分けが要らない

> **機械学習などの複雑な仕組みは使わずに、**記録の集計だけで作っている。やっているのは「数えて、割って、足す」だけ。

テーマ 03

終わりを先に決めておく



✓1つ目

はじめるための準備

✓2つ目

メインタスク

✓3つ目

おまけのタスク

「おわりの目安」を示すことで、完了を実感できる。どこで終わっていいかを、利用者が自分で決めなくて済む。

テーマ 04

明日もう一度ひらいてもらう

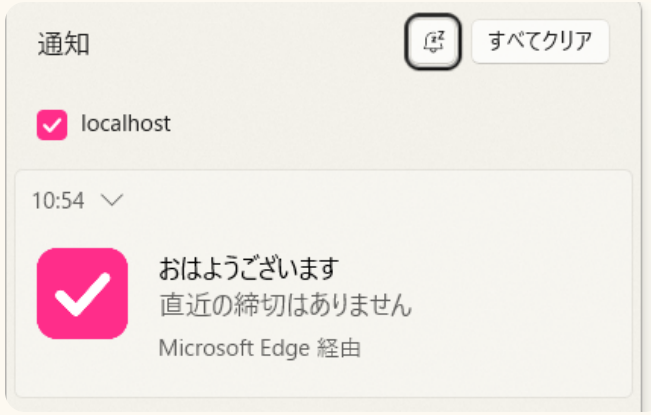
声をかける（ブラウザ通知）

種類	いつ	内容
朝の声かけ	毎朝8時	残りの件数とスタートへの入口
締切	近いとき	対象のタスク名
おまかせ完了	予約時刻	自動で完了したお知らせ

終わったことを分かりやすく

- 完了すると効果音（自作の合成音4種）と紙ふぶき
- どちらも設定で切れる





フォルダ構成

役割ごとに1階層だけ

```
com.example.taskboard
 ├── ai/         AI とのやりとり
 ├── config/     認証・CSRF / 鍵
 ├── controller/ 画面の入口（6つ）
 ├── domain/     テーブルに対応
 ├── dto/        画面用の入れ物
 ├── logic/      アプリ側の判断
 ├── repository/ DB出し入れ
 └── service/    処理のまとまり
```

言いたいことは3つ

- **ai/ は外との窓口。**Geminiを呼ぶのはこの中だけなので、AIを変えても他は触らない
- **logic/ はAIに任せない判断。**分野と時間はここで決めてからAIに渡す
- **上から下へ一方通行。**controller → service → repository → domain の向きだけ

データベース構成

テーブルは5つ

テーブル	なぜ必要か
users	人。誰の設定・誰のタスクかを決める
tasks	やること。画面に並ぶのはこれ
proposal_sets	提案1回ぶん。いつ押して、いつ選んだか。送った文章も保存する
proposal_cards	提案カード。1回に3枚。どれが選ばれたかを数えるため親と分けた
push_subscriptions	通知の届け先。端末ごとに1行

技術構成

Java と Spring Boot だけで組む

画面
Thymeleaf / 素のJavaScript

アプリ
Spring Boot 4.0.8 / Java 21

データベース
MySQL 9.7（本番） / H2（開発）

外部サービス
Gemini API / Google OAuth / Web Push

選定の方針

- **JavaScriptのフレームワークを使わない。**やることが並べ替えと削除と通知の購読だけなので、覚える量のほうが多くなる
- **画面はサーバー側で描く。**フォーム送信を基本にすると、処理の流れが目で追える
- **開発と本番でDBを分ける。**H2は起動が速いので、動作確認が速く回る

> 外部ライブラリで増やしたのは**通知まわりだけ**。紙ふぶきもcanvasに自前で描き、開閉はCSSだけで作った。

確かめ方

手で動かす確認と、自動テスト

手で確認したこと

- 初回ログインで行が作られる／2回目は作られない
- 3案が返り、重さが範囲内に収まる
- AIが失敗しても3案が出る
- 通知：許可→届く／切る→届け先が消える
- 他人のタスクを操作できない（IDを書き換えてPOST）
- 空・長すぎる入力・過去の締切は、注意を出して止まる

自動テスト（JUnit 5）　6クラス　合計27件

クラス	結果	確かめていること
SituationJudge	× 1件	決めたとおりの分野と時間が選ばれるか

失敗した1件は **SituationJudge**。分野の選び方には乱数が入るため、時間帯と設定を変えて100回くり返し、**「別の案を見る」で外した分野が、次の1件目に出ないか**を確かめた。
結果、条件によっては同じ分野がまた1件目に出ていた。点数を下げる形をやめ、候補から取り除く形に直した。

残している課題と、これから

残っている課題

- 外部サービスにタスクの中身を送らない形にする
- 端末ごとに通知の登録が要る（Web Push の仕組み上）
- プロンプトの改善による、AI提案の精度向上

これからやること

- 画面の動きを増やして、使い心地を上げる（React の導入）
- 完了率の反映を、記録を溜めて検証する
- テストの範囲を、画面（HTTP）まで広げる

押すことと選ぶことだけで回る

AIは文章を書くだけで、何を出すかはアプリが決めている

迷っているより、まず実行。ほんの少しでも。